Report on

AGENTIC DATA ANALYSIS ASSISTANT

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Electronics and Telecommunication Engineering

by

ANJALI KUMARI	2304008
AYUSH RAJ	2304018
ANKIT DASH	2304009

SCHOOL OF ELECTRONICS ENGINEERING

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

(Deemed to be University)

Bhubaneswar – 751024, Odisha, India

1. Title and Group Details

Project Title: Agentic Data Analysis Assistant

The Agentic Data Analysis Assistant is a tool-using application that accepts a CSV dataset and a natural-language analysis request. It uses a Hugging Face instruction-following language model through LangChain to interpret the request and select suitable tools from a controlled registry. Pandas performs data inspection and statistical computation, while Matplotlib generates visualisations. The agent is restricted to approved operations such as dataset inspection, descriptive statistics, categorical analysis, date analysis, filtering, group analysis and plot generation. The language model proposes tool calls; a deterministic validation and execution layer checks each operation and its arguments before running it.

Group Member	Roll Number	Contribution
Anjali Kumari	2304008	Frontend development using Streamlit, dashboard design, dataset visualization, data exploration and frontend-backend integration.
Ayush Raj	2304018	Backend development, LLM integration, LangGraph agent workflow and analytical tool implementation.
Ankit Dash	2304009	Project research, testing, documentation, result analysis and report preparation.

2. Problem Statement and Objectives

2.1 Description of the Selected Problem

CSV files are widely used to store and exchange tabular data, but extracting useful information often requires users to understand the dataset, choose statistical operations, filter or group records, and select suitable plots. These steps can be difficult for people without programming experience. A conversational agent can simplify the workflow by

translating a user's request into a sequence of data-analysis tool calls. However, an agent must not be allowed to execute arbitrary generated code. This project addresses both needs by connecting a language model to a fixed set of approved analysis functions, with validation before execution.

2.2 Motivation

The motivation is to make exploratory data analysis accessible through natural-language interaction while retaining control over computation. The language model handles request interpretation and tool selection, whereas Pandas and Matplotlib carry out the actual analysis. This separation aims to combine ease of use with repeatable calculations, transparent tool execution and a safer execution boundary.

2.3 Objectives

- Read and validate CSV datasets and inspect rows, columns, data types and missing values.
- Use the Qwen2.5-7B-Instruct model through LangChain to interpret requests and select tools from the approved registry.
- Compute descriptive statistics, categorical summaries, date analysis, filtering and grouped analysis using Pandas.
- Generate appropriate, clearly labelled plots using Matplotlib.
- Explain computed results and reject unsupported or unsafe operations through the restricted executor.
- Develop an interactive frontend that allows users to upload and explore CSV datasets.
- Establish communication between the frontend and backend through REST APIs.
- Maintain a conversational interface for submitting multiple analytical queries.
- Provide meaningful analytical results and visual representations to improve data understanding.
- Develop a modular architecture that allows independent modification of frontend and backend components.

•

2.4 Expected Outcomes

The completed system provides a conversational interface for CSV analysis. It inspects a dataset, interprets a user's request, chooses one or more approved tools, executes the corresponding Pandas operation, generates Matplotlib plots where appropriate, and returns a concise explanation with the computed output. It is designed to reject unsupported requests and invalid tool parameters instead of executing arbitrary code.

3. Proposed Methodology and Processing Pipeline

3.1 Block Diagram of the Complete Processing Pipeline

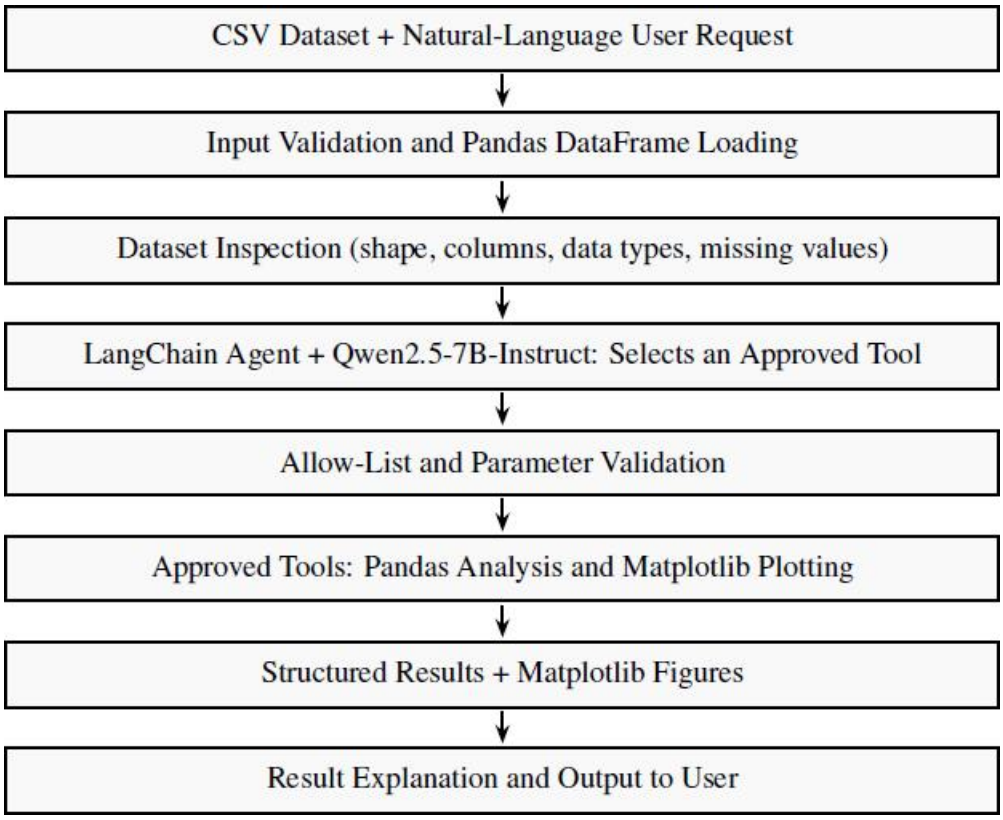


Figure 1: Proposed processing pipeline of the Agentic Data Analysis Assistant.

3.2 Step-by-Step Processing Stages

- Input: The user provides a CSV file and enters a natural-language request, such as asking for summary statistics or a chart.
- File validation: The application checks that the input is a readable CSV file and handles parsing errors with a clear message.
- Data loading and inspection: Pandas loads the file into a DataFrame. The system inspects the dataset dimensions, column names, data types and missing-value counts.
- Request interpretation: The LangChain agent uses Qwen2.5-7B-Instruct to identify the user's intent and select one or more functions from the approved tool registry.

- Safety validation: The operation name, selected columns, data types and parameters are checked against the tool schema and allow-list before execution.
- Analysis: The selected Pandas tool computes descriptive statistics, categorical frequencies, date summaries, filtered records or grouped results as requested.
- Visualisation: The generate_plot() tool uses Matplotlib to create an appropriate histogram, bar chart, scatter plot, line chart or other supported visualisation.
- Output: The application presents the structured tool results, plot and a concise explanation grounded in the computed output.

3.3 Input, Intermediate Processing and Output

Stage	**Description**
Input	CSV dataset and a natural-language analysis request.
Intermediate processing	CSV validation, DataFrame creation, schema inspection, operation planning, allow-list validation and Pandas computation.
Output	Descriptive statistics, data-quality summaries, Matplotlib plots, explanatory text and error messages for unsupported requests.

4. Models and Tools Used

4.1 ML / NLP / LLM Models

The project uses a pretrained instruction-following language model as the natural-language reasoning and tool-selection component. The selected model in the implementation workflow is Qwen/Qwen2.5-7B-Instruct, accessed through Hugging Face Inference Providers and integrated with LangChain. The model is not fine-tuned for this project. It interprets user requests and selects registered tools; it does not directly calculate dataset statistics or generate plots.

Pretrained model: Qwen/Qwen2.5-7B-Instruct. Hugging Face model page: https://huggingface.co/Qwen/Qwen2.5-7B-Instruct. Inference access: Hugging Face Inference Providers. The selected model/provider must support the required LangChain tool-calling interface in the deployed configuration.

4.2 Libraries and Frameworks

Tool / Library	**Purpose**
Python 3	Application logic and approved-operation executor.
Pandas	CSV loading, DataFrame handling, data inspection and statistical analysis.
Matplotlib	Generation of labelled charts and plots.
NumPy	Numerical operations where required.
LangChain	Agent orchestration and connection of the language model to approved tools.
langchain-huggingface	Hugging Face chat model integration for LangChain.
huggingface_hub	Hugging Face authentication and Inference Providers access.
Google Colab	Notebook execution and secure storage of HF_TOKEN using Colab Secrets.

4.3 Implementation Environment

The application is developed in Python and demonstrated through Google Colab. The Hugging Face access token is stored in Colab Secrets under the name HF_TOKEN, with notebook access enabled. The token is retrieved at runtime and is never hard-coded into the notebook or shared in source control. The selected integration uses langchain-huggingface and huggingface_hub with Pandas, Matplotlib, NumPy and LangChain. Authentication must be verified with HfApi.whoami(); successful model initialisation must then be followed by a real inference test, since endpoint construction alone does not prove that the provider call works.

5. Implementation and Usability

5.1 Implementation Procedure

- Configure the approved tools: inspect_dataset(), calculate_statistics(), analyze_categorical(), analyze_dates(), filter_data(), group_analysis() and generate_plot().
- Implement CSV input handling, file validation and Pandas DataFrame loading.
- Load the dataset with Pandas and generate a schema and data-quality summary.
- Create a fixed registry mapping approved tool names to predefined Python functions and input schemas.
- Configure HuggingFaceEndpoint for Qwen/Qwen2.5-7B-Instruct and wrap it with ChatHuggingFace to create the LangChain chat model.
- Connect the model to the approved LangChain tools. Validate every operation and parameter before execution; never execute arbitrary model-generated code.
- Implement the approved statistical, filtering, grouping and Matplotlib plotting functions.
- Integrate the model, LangChain tool interface, allow-list validator, Pandas executor, Matplotlib functions and response formatter.
- Test valid requests, invalid columns, malformed CSV files, unavailable inference, and prohibited operation requests.

5.2 Instructions for Running the Application

The following procedure describes the Google Colab workflow used for the project.

- Open the completed project notebook in Google Colab.
- In the Colab Secrets panel, create a secret named HF_TOKEN, paste the Hugging Face token as its value, and enable notebook access.
- Install the Hugging Face integration packages with %pip install -q -U langchain-huggingface huggingface_hub. Install Pandas, Matplotlib, NumPy and LangChain dependencies as listed by the project.
- Retrieve the token securely with userdata.get('HF_TOKEN') and verify authentication using HfApi(token=hf_token).whoami(). Never print or hard-code the token.
- Initialise HuggingFaceEndpoint with repo_id='Qwen/Qwen2.5-7B-Instruct', task='text-generation' and the retrieved token; wrap it using ChatHuggingFace.
- Connect the chat model to the approved LangChain tools, load a CSV file, submit a supported request and review the statistics, plot and explanation.

5.3 Practical Applications and Usability

The assistant can be used by students learning statistics and Python, educators demonstrating exploratory data analysis, and analysts who need quick summaries of tabular data. Example applications include survey analysis, academic records, sales summaries, sensor datasets and date-based trend inspection. Users should have permission to process the selected data.

6. Results and Sample Outputs

6.1 Representative Input

The Agentic Data Analysis Assistant accepts structured CSV datasets and natural-language queries from users. The input dataset contains tabular information arranged in rows and columns, which is loaded and processed using the Pandas library. After uploading the dataset, the user can provide instructions in ordinary English without manually writing Python code. The system is designed to interpret the user's request, identify the required analytical operation and select an appropriate tool from the predefined tool registry.

Several representative queries can be used to evaluate the functionality of the proposed system. For example, a user can ask, "Inspect the uploaded dataset and display its column names, data types and missing values." This request requires the dataset inspection tool. Similarly, the query "Calculate descriptive statistics for all numerical columns" requires the statistical analysis tool. A categorical analysis request such as "Identify the category with the highest total sales" requires grouping and aggregation of the selected categorical column. The user can also request filtering operations, such as "Display records where Sales is greater than 500", or visualization operations, such as "Generate a histogram for the Sales column."

These representative inputs cover the major analytical capabilities of the application, including dataset inspection, descriptive statistics, categorical analysis, filtering, grouping and graphical visualization. They also demonstrate how natural-language interaction can simplify the process of exploratory data analysis.

6.2 Corresponding Output

For a request to calculate statistics, the system returns a structured

summary for numerical columns, including count, mean, standard deviation, minimum, quartiles and maximum. For categorical analysis it returns category counts or frequencies. For date analysis it summarises valid date ranges and supported time groupings. For a plotting request it returns a Matplotlib figure with a title and labelled axes. The final language-model response explains the returned values without inventing statistics.

Illustrative example: if a numeric test column contains 10, 20, 30, 40 and 50, the Pandas statistics output has count 5, mean 30, minimum 10 and maximum 50. This is a simple demonstration dataset, not a benchmark result from a real-world dataset.

6.3 Screenshots and Generated Responses

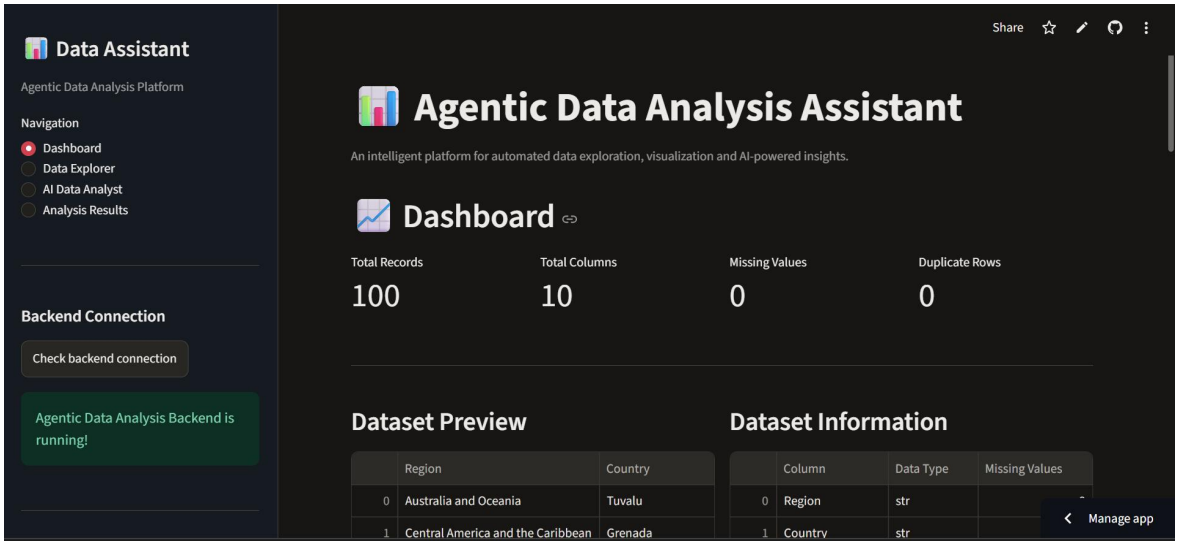


Fig-1 Apps dashboard

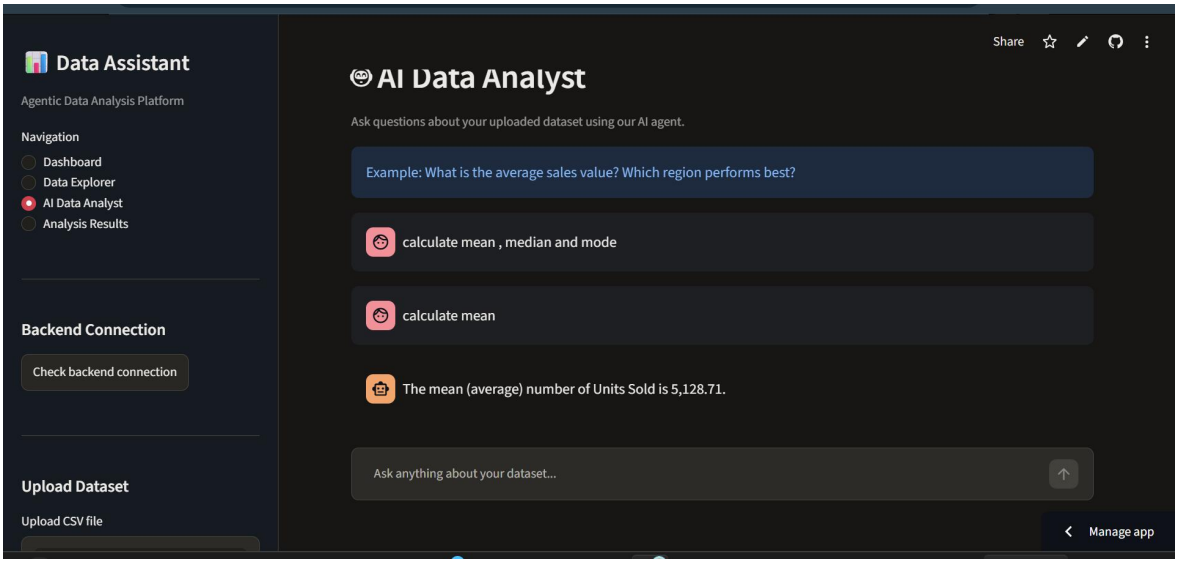


Fig 2- mean calculated

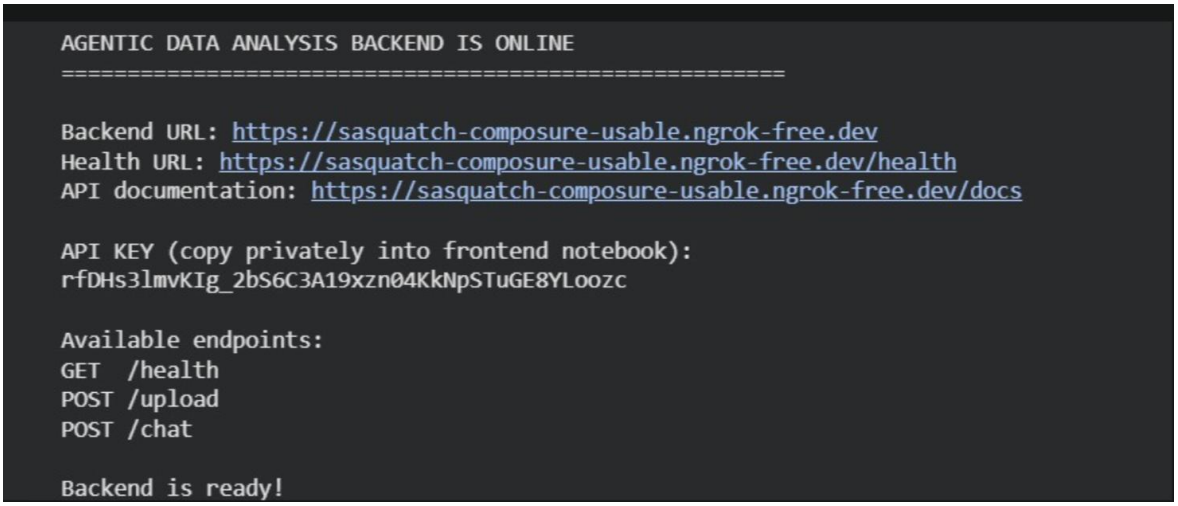


Fig 3 - our backend url and api



Fig 4- registered tools

6.4 Performance Evaluation and Observations

The evaluation procedure compares Pandas results with independently calculated expected values, checks whether Qwen selects the correct tool for each request, inspects plot labels and readability, and verifies that invalid or unapproved operations are rejected. Hugging Face authentication is checked separately with HfApi.whoami(). The

supplied development evidence showed an HTTP 401 Unauthorized response during inference and authentication verification; therefore, the final report does not claim that live inference was successfully completed until the HF_TOKEN secret is replaced with a valid Inference Providers token and the model test returns a response. Numerical correctness and tool-safety tests should be recorded from the completed run.

6.5 Limitations

The system is limited to CSV input and the analysis operations implemented in its approved registry. The live LLM component depends on Hugging Face Inference Provider availability, account permissions, service availability and rate limits. During development, the inference request returned HTTP 401 Unauthorized, so successful live model execution must be confirmed after correcting the Colab secret. The model may misunderstand ambiguous requests or select an unsuitable tool; the validator remains essential. Large datasets may require additional memory and execution-time controls. Statistical summaries depend on source-data quality, and correlation does not establish causation.

7. Conclusion and Future Scope

7.1 Conclusion

The Agentic Data Analysis Assistant is designed to simplify exploratory data analysis through natural-language interaction. The project combines a pretrained Qwen instruction-following language model with LangChain and LangGraph for agent-based tool orchestration.

Pandas and NumPy are used for data processing and statistical calculations, while visualization libraries provide graphical representations of analytical results. The Streamlit frontend offers an interactive interface for dataset uploading, exploration and conversational analysis. FastAPI provides backend endpoints to connect the frontend with the AI analysis system.

The project demonstrates how Large Language Models and predefined analytical tools can be combined to develop an accessible data-analysis application. It reduces the need for users to manually write analytical code and provides a structured approach to exploring datasets.

7.1 Future Scope

- Add support for Excel, JSON and Parquet datasets.
- Include more approved operations, such as group-wise analysis, outlier inspection and time-series summaries.
- Add export of analysis reports to PDF, HTML or Excel.
- Improve request interpretation through evaluation of alternative instruction models and inference providers, and consider a locally hosted model for privacy-sensitive use cases.
- Introduce user authentication, audit logs, dataset-size controls and stronger privacy options.
- Conduct larger usability tests and report quantitative measures such as

calculation correctness, tool-selection accuracy, rejection of unsafe requests and response time.

8. References and GitHub Repository Link

[1] Qwen Team, "Qwen2.5: A Series of Large Language Models," Hugging Face Model Repository. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-3B-Instruct

[2] Hugging Face, "Transformers Documentation." [Online]. Available: https://huggingface.co/docs/transformers/

[3] LangChain, "LangChain Documentation." [Online]. Available: https://python.langchain.com/docs/

[4] LangGraph, "LangGraph Documentation." [Online]. Available: https://langchain-ai.github.io/langgraph/

[5] Streamlit, "Streamlit Documentation." [Online]. Available: https://docs.streamlit.io/

[6] FastAPI, "FastAPI Documentation." [Online]. Available: https://fastapi.tiangolo.com/

[7] Pyngrok, "Pyngrok Documentation." [Online]. Available: https://pyngrok.readthedocs.io/

[8] GitHub Repository URL: https://github.com/Anjalikumari526/Agentic-Data-Analysis-Assistant

(our github link is above , copy and paste it to broswer to access the github)